Myntra — Cloud Scaling for Flash Sales and Personalization

Week 2 Report: Case Study Analysis, Architecture, Exercise Plan and Team Roles

Cloud Computing — Group 33

November 2026

Member	Roll No.	Role
Utsker Dhameliya	202301083	Research and Architecture
Ritul Patel	202301086	Application Development
Divyesh Vagh	202301444	Cloud Infrastructure

1. Introduction

1.1 Background

Myntra is one of India's largest fashion e-commerce platforms, founded in 2007 and acquired by Flipkart (Walmart group) in 2014. The platform serves tens of millions of customers across India, offering products from thousands of brands across apparel, footwear, accessories and lifestyle categories [V1].

Myntra's technology has evolved from a monolithic PHP and MySQL architecture into a distributed, service-oriented platform running on Microsoft Azure. This evolution was driven by the need to handle extreme traffic spikes during flash-sale events while simultaneously delivering personalized shopping experiences through real-time machine-learning systems.

1.2 Project Motivation

This case study examines how Myntra addresses two of cloud computing's most demanding challenges:

1. Elastic scalability — handling traffic spikes of 15–20× normal levels during major sales events, where fixed infrastructure would be prohibitively expensive and insufficient.

2. Real-time data processing at scale — ingesting millions of events per minute, computing personalization features with sub-millisecond latency, and serving 2.5 PB of data daily during peak events.

1.3 Case-Study Objectives

1. Analyse Myntra's documented cloud architecture and engineering solutions.

2. Study the specific problem of flash-sale traffic scaling.

3. Evaluate Myntra's data and personalization architecture.

4. Design a practical exercise that demonstrates cloud-scaling principles.

5. Develop a reproducible experimental methodology for measuring latency, scaling behaviour and cost.

1.4 Scope

This report covers the case analysis and architecture study (Week 2). It does not include implementation or experimental results, which are planned for Week 3. All claims about Myntra's architecture are based on publicly available engineering blog posts, vendor case studies and conference presentations. Proposed metrics and cost estimates are labelled as hypotheses or estimates, not measured outcomes.

2. Problem Statement

2.1 The Flash-Sale Challenge

Myntra's End of Reason Sale (EORS), held approximately twice a year, generates traffic spikes of 15–20× normal levels [P3]. This creates a fundamental infrastructure challenge: the system must handle extreme short-duration demand without failing, while avoiding the cost of permanently provisioning peak capacity.

2.2 Traffic Characteristics

Metric	Value	Source	Year
EORS traffic multiplier	15–20× normal	Myntra Engineering [P3]	2020–2021
Order-handling target	12,000 orders/min	Myntra Engineering [P3]	2019
Order-handling capacity	20,000 orders/min	Myntra Engineering [P3]	2020
BFF visits	~460 million	Myntra reports	2023
BFF visits	~627 million	Myntra reports	2024

2.3 Performance Challenges

Discount service latency: Computing and serving product discounts at ~150 ms latency before optimization [P2].

Feature store access: Personalization feature lookups at 8.5 ms, too slow for millions of concurrent users [V2].

Event ingestion volume: Approximately 4 million events/minute through Kafka-based pipelines [P4].

2.4 Infrastructure Challenges

Challenge	Impact
Fixed peak-capacity provisioning	Extremely high cost; resources idle most of the year
Vertical scaling limits	Single servers reach CPU/memory ceilings
Single points of failure	One failing service cascades to entire platform
Long provisioning cycles	Physical hardware cannot be deployed in minutes
Regional outages	Single-region deployment risks total unavailability

2.5 The Core Trade-Off

The fundamental problem is the trade-off between cost and capacity. Provisioning for peak (3 VMs always) costs ~₹360/day for our exercise, but ~47% of compute is wasted during normal traffic. Elastic provisioning (1–3 VMs) costs ~₹192/day, saving compute while still meeting peak demand.

3. Research Findings

3.1 Verified Metrics

Metric	Value	Source	Year	Context
EORS traffic spike	15–20× normal	[P3]	2020–2021	EORS event traffic
Order capacity	20,000 orders/min	[P3]	2020	EORS planned capacity
Discount latency	150 ms → ~30 ms	[P2]	2020–2021	Discount service
Feature lookup	8.5 ms → 0.8 ms	[V2]	2022–2023	Online feature store
Feature operations	500K ops/sec	[V2]	2022–2023	Feature store throughput
Event ingestion	~4M events/min	[P4]	2019–2020	Kafka ingestion
Data serving (BFF)	2.5 PB/day	[P1]	2022	BFF event data
Analytical queries	65,000+/day	[P1]	2022	BFF workload
Data infra cost	35% reduction	[V3]	2023–2024	Data platform
Pipeline speed	25% improvement	[V3]	2023–2024	Data processing
EORS order growth	50% increase	[V1]	2020	EORS 2020

3.2 Source Assessment

Source	Type	Reliability
Myntra Engineering [P1–P4]	First-party engineering blog	High
Microsoft [V1]	Vendor partner case study	Medium–High
Aerospike [V2]	Vendor partner technical	Medium–High
Databricks [V3]	Vendor partner technical	Medium–High

3.3 Important Caveats

1. Temporal separation: These metrics span 2019–2024 and represent different systems and workloads. They should not be treated as a single before-and-after comparison.

2. Workload specificity: Cost reductions and performance improvements apply to specific subsystems, not the entire Myntra infrastructure.

3. Attribution: The 50% EORS order growth in 2020 coincided with pandemic-driven e-commerce growth and cannot be solely attributed to the Azure migration.

4. Myntra's Cloud Architecture

4.1 Azure Migration

Myntra announced its migration to Microsoft Azure in February 2020 [V1], motivated by elastic infrastructure, managed services, multi-region capability and global availability.

4.2 Documented Infrastructure

Application layer:

- Service-oriented architecture with independent scaling per service
- API gateway for request routing
- Product, Cart, Order, Search and User services

Resilience mechanisms:

- Load testing before every EORS event [P3]
- Autoscaling based on demand metrics
- Rate limiting and circuit breakers
- Kafka capacity scaling during peak events

Disaster recovery:

- Dual-region deployment on Azure [P3]
- Data replication between regions
- Regular DR drills
- Traffic management for regional failover

4.3 Production Architecture Diagram

Label: "Conceptual reconstruction of publicly documented Myntra architecture."

The production architecture diagram illustrates the relationships between: customers → load balancing → application services → pricing (Redis/MongoDB) → event ingestion (Debezium/Sourcerer/Kafka) → processing (Quicksilver/Janus) → data lakehouse (Bifrost) → personalization (Feature Store/Aerospike/Recommendations). Dual-region disaster recovery spans the infrastructure.

See: week2/diagrams/production-architecture.mmd for the Mermaid source.

5. Data and Personalization Architecture

5.1 Sourcerer — Event Ingestion

Sourcerer is Myntra's data ingestion system built on Debezium and Kafka [P4]. Debezium captures database changes (INSERT, UPDATE, DELETE) from transaction logs using Change Data Capture (CDC). Sourcerer manages Debezium connectors, handles schema evolution and routes events to Kafka topics. Kafka serves as the central event bus, processing approximately 4 million events per minute.

5.2 Quicksilver — Near-Real-Time Processing

Quicksilver consumes events from Kafka and performs near-real-time processing [P1]: stream processing, event enrichment, transformation and real-time aggregation for dashboards.

5.3 Janus — Data Quality

Janus is involved in data quality and validation within the pipeline [P1], ensuring events meet expected schemas before reaching downstream consumers. Note: The exact scope of Janus is less extensively documented than other components.

5.4 Bifrost — Data Lakehouse

Bifrost is Myntra's unified data lakehouse [P1], combining data lake flexibility with data warehouse performance. During BFF 2022: 2.5 PB/day of data serving and 65,000+ analytical queries/day.

5.5 Feature Store and Aerospike

Layer	Technology	Purpose	Performance
Online	Aerospike	Real-time feature serving	0.8 ms lookup, 500K ops/sec
Offline	Bifrost	Batch feature computation	Batch processing

The Aerospike deployment reduced feature-lookup latency from 8.5 ms to 0.8 ms — a 10× improvement [V2].

5.6 Recommendation Pipeline

1. Capture: Clickstream and behavioral data → Kafka
2. Process: Kafka → Quicksilver (real-time feature updates)
3. Store: Quicksilver → Bifrost (offline) + Aerospike (online)
4. Serve: Recommendation engine queries Aerospike → ML models rank products
5. Deliver: Personalized results returned within page-load latency budgets

5.7 Component Timeline

Component	Source	Approx. Date
Sourcerer / Debezium	[P4]	2019–2020
Azure migration	[V1]	February 2020
BCP/DR, EORS scaling	[P3]	2020–2021

Pricing optimization	[P2]	2020–2021
Bifrost, Quicksilver	[P1]	2022–2023
Aerospike feature store	[V2]	2022–2023
Data cost optimization	[V3]	2023–2024

Important: These components were documented at different times and may represent different architectural iterations.

6. Technical Case Analysis

6.1 Scalability

- Horizontal scaling: Independent services scale based on their load profiles [P3].
- Kafka partition scaling: Capacity increases during peak events, then scales back.
- Autoscaling: Cloud resources scale automatically based on demand metrics.

6.2 Resilience

- Load testing before every major sale event [P3]
- Rate limiting to protect backend services
- Circuit breakers to prevent cascading failures
- Dual-region disaster recovery [P3]

6.3 Caching and Pricing Architecture

Before	After	Technique
~150 ms latency	~30 ms latency	Redis read balancing
Single read path	Multiple Redis replicas	Read distribution
No fallback	MongoDB persistence	Graceful degradation

This 5× latency improvement is significant at scale: with millions of concurrent page views during EORS, saving 120 ms per discount lookup compounds into massive reductions in total compute.

6.4 Disaster Recovery

Failure Mode	Protection
Service instance failure	Autoscaling and health checks
Availability zone failure	Distribution across zones
Regional outage	Dual-region with traffic failover
Database corruption	Replicated data stores

6.5 Cost Optimization

Strategy	Mechanism	Impact
Elastic compute	Autoscale based on demand	Avoid idle peak capacity
Efficient data infra	Bifrost, optimized pipelines	35% lower costs [V3]
Caching	Redis reduces database load	Fewer compute queries
Pipeline optimization	Faster processing	25% speed improvement [V3]

7. Proposed Solution: MyntraFlash

7.1 Application Overview

MyntraFlash is a scaled-down e-commerce simulation inspired by Myntra, designed to demonstrate cloud-scaling principles in the Week 3 practical exercise.

7.2 Technology Stack

Component	Technology	Justification
Cloud Provider	Microsoft Azure	Aligns with Myntra [V1]
Backend	Node.js + Express	Lightweight, stateless
Frontend	HTML, CSS, JavaScript	Simple, no build tools
Containerization	Docker	Consistent deployment
Infrastructure	Azure VM Scale Set	Managed autoscaling
Load Balancing	Azure Standard LB	Health probes, L4
Autoscaling	Azure Monitor	CPU-based rules
Load Testing	Grafana k6	Scriptable, open-source

7.3 Application Endpoints

Endpoint	Purpose
GET /products	Product catalogue
GET /flash-sale	Flash-sale items with countdown
GET /api/discount	Discount calculations
GET /api/recommend	Recommendations
GET /health	Health check for LB probes
GET /instance-id	Instance identification

7.4 Stateless Design

The application is completely stateless — no server-side sessions, no local file state, no shared mutable state. This enables round-robin load balancing, instant instance utility, safe scale-in and simple fault tolerance.

7.5 Proposed Architecture Diagram

The proposed architecture diagram shows: k6 Load Generator → Azure Standard Load Balancer → VM Scale Set (1–3 instances, each running Docker with MyntraFlash) ← Azure Monitor Autoscale (CPU > 70% scale-out, CPU < 30% scale-in, 5-min cooldown).

See: week2/diagrams/proposed-architecture.mmd for the Mermaid source.

7.6 Infrastructure Configuration

Resource	Configuration
VM Size	Standard_B2s (2 vCPU, 4 GB RAM)
OS	Ubuntu 22.04 LTS
VMSS min/max	1 / 3 instances
Scale-out	CPU > 70% for 5 min → +1
Scale-in	CPU < 30% for 5 min → −1

Cooldown	5 minutes
Health probe	HTTP /health, 15s interval
Load balancer	Standard, TCP 80 → 3000

8. Exercise and Experimental Plan

8.1 Experiments Overview

#	Experiment	Objective	Course Req.
1	Baseline performance	Single-VM metrics	Latency
2	Horizontal scaling	1 vs. 2 vs. 3 VMs	Scaling
3	Autoscaling	CPU-triggered scale-out/in	Scaling
4	Fault tolerance	Health probe failover	Resilience
5	Cost analysis	Fixed vs. elastic cost	Cost

8.2 Experiment 1 — Baseline

Deploy with 1 VM (autoscaling disabled). k6 with 50 VUs for 5 minutes.

Metrics: average response time, p50/p95/p99 latency, requests/sec, error rate, CPU utilization.

8.3 Experiment 2 — Horizontal Scaling

Identical load (100 VUs, 5 min) against 1, 2 and 3 VMs. Same image, VM size, region, endpoint mix.

Expected: Latency decreases and throughput increases with instance count.

8.4 Experiment 3 — Autoscaling

Enable autoscale (min 1, max 3). Ramp from 20 → 200 → 500 → 20 VUs over ~34 minutes. Observe scale-out timing, latency improvement, scale-in behaviour and cooldown effects.

8.5 Experiment 4 — Fault Tolerance

Deploy 2 instances. Stop Docker on one VM. Observe health probe detection, traffic rerouting and recovery when container restarts.

8.6 Experiment 5 — Cost Analysis

Scenario	Est. 24h Cost	VM-Hours
Fixed (3 VMs always)	~₹360	72
Elastic (1–3 VMs)	~₹192	30
Savings	~₹168 (~47%)	42 hours saved

These are estimates based on Azure Pricing Calculator. Actual costs will be measured in Week 3.

9. Team Roles and Project Execution

9.1 Responsibility Matrix

Responsibility	Utsker	Ritul	Divyesh
Case-study research	Lead	Support	Support
Source verification	Lead	—	—
Production architecture	Lead	—	Support
App development	—	Lead	—
Docker implementation	—	Lead	Support
Azure infrastructure	—	—	Lead
Autoscaling & monitoring	—	—	Lead
Load testing	—	—	Lead
Results analysis	Lead	Support	Support
Report writing	Lead	Support	Support
Viva preparation	All	All	All

9.2 Week 3 Timeline

Day	Activity	Owner
Day 1–2	Implement Node.js app and Docker	Ritul
Day 2–3	Provision Azure infrastructure	Divyesh
Day 3	Deploy application to VMSS	Divyesh + Ritul
Day 4	Run Experiments 1–3	Divyesh
Day 5	Run Experiments 4–5	Divyesh
Day 5–6	Analyse results	All
Day 6–7	Document findings	Utsker

9.3 Risks and Mitigation

Risk	Mitigation
Azure spending limits	Use student subscription; set budget alerts
Autoscaling doesn't trigger	Lower CPU threshold; increase load
Application bugs	Test locally with Docker before Azure
Time pressure	Prioritize Experiments 1–3

10. Expected Outcomes and Conclusion

10.1 Expected Learning Outcomes

1. Practical cloud deployment: Hands-on experience with Azure VMSS, Load Balancer and Monitor.
2. Horizontal scaling: Measured evidence of performance improvement with additional instances.
3. Autoscaling: Understanding of CPU-based triggers, cooldowns and scaling dynamics.
4. Fault tolerance: Demonstration of health-probe-based traffic rerouting.
5. Cost optimization: Quantified comparison of fixed vs. elastic provisioning.

10.2 Evaluation Metrics

Category	Metrics
Latency	Average, p50, p95, p99 response time (ms)
Throughput	Requests per second
Reliability	Error rate (%)
Scalability	Latency vs. instance count curve
Autoscaling	Scale-out/in event timing and CPU
Cost	VM-hours, total cost for fixed vs. elastic

10.3 Limitations

1. Scale difference: Our exercise uses 1–3 VMs; Myntra uses thousands of instances.
2. Simplified workload: MyntraFlash uses in-memory data; Myntra has distributed databases.
3. No data architecture: The exercise does not implement Kafka, Bifrost or feature stores.
4. Single region: Myntra uses multi-region DR.
5. Burstable VMs: B2s VMs may throttle under sustained high load.
6. Network effects: Single test machine vs. geographically distributed real users.

10.4 Conclusion

Myntra's architecture demonstrates how cloud elasticity, distributed event processing, resilient infrastructure and low-latency caching work together to support flash-sale traffic while delivering personalized recommendations. The documented improvements — 5× discount-service latency reduction, 10× feature-lookup improvement, 35% data infrastructure cost savings — represent specific, verifiable outcomes across individual subsystems.

Our MyntraFlash exercise distils these principles into a practical, measurable demonstration. By deploying a stateless application on Azure VMSS with load balancing and autoscaling, then subjecting it to simulated flash-sale traffic, we can directly measure the latency, throughput and cost implications of elastic versus fixed infrastructure — the same fundamental trade-off that drove Myntra's cloud migration.

References

ID	Source	Key Information	Date
[P1]	Myntra Engineering —	Bifrost, Quicksilver, 2.5 PB/day, 65K+	2022–2023

	Data Platform	queries/day	
[P2]	Myntra Engineering — Pricing System	Discount latency 150 ms → ~30 ms	2020–2021
[P3]	Myntra Engineering — BCP/DR	EORS 15–20× traffic, dual-region DR	2020–2021
[P4]	Myntra Engineering — Sourcerer	Debezium CDC, Kafka, ~4M events/min	2019–2020
[V1]	Microsoft — Azure Migration	Myntra on Azure, 50% EORS order growth	Feb 2020
[V2]	Aerospike — Customer Case Study	Feature lookup 8.5→0.8 ms, 500K ops/sec	2022–2023
[V3]	Databricks — Case Study	35% lower data costs, 25% faster pipelines	2023–2024
[A1]	Microsoft Learn — VMSS	learn.microsoft.com/azure/virtual-machine-scale-sets	—
[A2]	Microsoft Learn — Autoscale	learn.microsoft.com/azure/azure-monitor/autoscale	—
[A3]	Microsoft Learn — Load Balancer	learn.microsoft.com/azure/load-balancer	—
[A6]	Azure Pricing Calculator	azure.microsoft.com/pricing/calculator/	—
[L1]	Grafana k6 Documentation	k6.io/docs/using-k6/metrics/	—